Benifix AI Agent Hackathon Plan

The theme is **"Building a Smart Benefits Assistant"** for a company called Benifix. Teams will start with the provided codebase and progressively enhance the AI agent's capabilities.

Core Tooling and Setup

The provided setup (`app.py`, `tools.py`, `dbsetup.sql`) already covers the foundation:

- **Backend:** A Flask API (`app.py`) backed by an AlloyDB instance (as simulated by `dbsetup.sql`).
- **Tooling:** An initial working tool (`get_support_ticket_status`) and two stubs (`find_providers`, `get_employee_benefits`).

Challenge 1: Foundational Tools (Build & Connect)

Goal

Teams must complete the two pending tools in `tools.py` and test them using the Gemini CLI.

Tasks

1. **Implement `find_providers`:** Complete the `requests.get()` call to the `/api/providers` endpoint, passing the `specialty` as the `query` parameter (as per the hint in the provided `app.py`).
2. **Implement `get_employee_benefits`:** Complete the `requests.get()` call to the `/api/benefits` endpoint, passing the `employee_id` as a parameter.
3. **Test & Combine:** Successfully answer a complex query that requires the agent to use *both* new tools (e.g., `"My ID is 123. What's my deductible, and can you find me a dentist?"`).

Datasets/DDL

The existing `dbsetup.sql` handles this. You have tables for `benefits` and `providers` with synthetic data.

Challenge 2: Advanced Data Handling & Semantic Search (AlloyDB Integration)

This challenge introduces a more complex, unstructured data scenario, leveraging a potential future or advanced feature of AlloyDB for search and **Vertex AI** for embeddings, fulfilling your desire to explore semantic challenges.

Goal

Teams must create a new tool that can answer questions about benefits *policies* (which are long-form, unstructured text documents) using a semantic search approach.

Tasks

1. **DB & Data Preparation:**
    - Add a new table (e.g., `policy_documents`) to the AlloyDB setup with columns for `policy_id`, `chunk_text`, and `embedding` (using a `VECTOR` type, if supported, or a large `TEXT` field for a simpler setup).
    - Create a synthetic dataset of "chunks" from a mock "Benifix Gold PPO Policy Document."
2. **Perceptive Modeling/Tool Integration:**
    - Create a new tool, `search_policy_docs(query: str) -> list[str]`.
    - The tool will *simulate* the following process:
        - Take the user's `query`.
        - Call an external API (or simulate a call) to a **Vertex AI Embeddings** model to get the vector for the `query`.
        - Execute a **SQL query** against the AlloyDB instance to find the top 3 closest `policy_documents` chunks based on vector similarity.
        - Return the text of the chunks.
    - *Self-Correction/RAG:* Teams can further challenge themselves by adding a second step where the agent sends the original query *and* the retrieved chunks to the Gemini model to synthesize a final answer.

Datasets/DDL

- **New DDL:** A `policy_documents` table with a `VECTOR` (or `TEXT`) field.
- **New Synthetic Data:** A file (e.g., `policy_data.sql`) with inserted chunks of policy text.

Challenge 3: Numerical & Predictive Analysis (Cost Forecast) 💰

Goal

Teams must create a new tool that predicts an employee's estimated annual out-of-pocket (OOP) healthcare costs based on their current plan and a set of usage assumptions. This involves numerical computation and conditional logic based on the data retrieved from AlloyDB.

Tasks

1. DB & Data Preparation (Optional Enhancement):
    - New DDL: You can optionally create a new table, `usage_profiles`, to store estimated annual visits or costs for different employee types (e.g., `low_user`, `average_user`, `high_user`). For simplicity, you can also hardcode these assumptions into the Python tool.
    - *Example Assumption:* A "High User" typically incurs $4,000 in claims before they hit their OOP max.
2. Create `predict_annual_cost` Tool:
    - Define a new tool: `@tool def predict_annual_cost(employee_id: str, usage_profile: str) -> dict:`.
    - Logic:
        - Step 1: Retrieve Benefits Data: The tool must use the `get_employee_benefits` tool (from Challenge 1) to fetch the `plan_name`, `individual_deductible`, and `individual_oop_max` for the given `employee_id`.
        - Step 2: Apply Numerical Logic: Based on the `usage_profile` (e.g., "High"), calculate the predicted cost:
            - *High Usage Scenario:* Assume the employee will hit their full Individual OOP Max.
            - *Average Usage Scenario:* Assume the employee will hit their Individual Deductible plus a fixed amount (e.g., $1,000) but stay below the OOP max.
        - Step 3: Prediction: Calculate the estimated annual out-of-pocket cost. Predicted Cost=MIN(Expected Claims−Deductible,Individual OOP Max)\text{Predicted Cost} = \text{MIN}(\text{Expected Claims} - \text{Deductible}, \text{Individual OOP Max})
3. Agent Prompt Engineering: Teams must test the agent's ability to chain the data retrieval (AlloyDB via API) and the numerical calculation (Python tool logic).

- *Example Query:* `"Based on my employee ID 123, I expect to be a 'high user'. What is my estimated out-of-pocket cost for the year?"`

Challenge 4: Proactive Notifications & Workflow (System Integration)

This challenge focuses on using the agent to interact with an *external* system (a common real-world task) and to make a decision based on data retrieved from AlloyDB.

Goal

Teams must create a tool that checks a user's benefits status and, if a specific condition is met, calls an external system to send a notification.

Tasks

1. **Create `trigger_alert` Tool:**
   - Define a new tool: `@tool def trigger_alert(employee_id: str) -> str:`.
   - The goal of this tool is to check if an employee's **Individual Deductible** is below a certain threshold (e.g., $500).
   - The tool logic:
     - Call the **AlloyDB API** (via `requests`) using the **`get_employee_benefits`** tool from Challenge 1.
     - Parse the returned JSON.
     - If `individual_deductible < 500`: Simulate a POST request to an external notification service (e.g., a simple `print()` statement or a mock `requests.post()` to a fake endpoint). Return a success message.
     - Otherwise, return a message that no alert was necessary.
2. **Agent Prompt Engineering:** Teams must now prompt the agent to decide *when* to use this new tool (e.g., `"Please check if employee 123 is eligible for a deductible alert."`).

Datasets/DDL

The existing `benefits` table data is sufficient, as it contains the `individual_deductible` field.

Challenge 5: Places API

API and API Key